

Lab 4: AV → NGAV → EPP → EDR → XDR

Detecció d'amengaces

GEI | FIB | Curs 2026/2027



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona

Detecció d'amengaces

Assignatura amb 22 hores de laboratori corresponent a l'especialitat:

Tecnologies de la informació

Incloua en el:

Grau en Enginyeria Informàtica



Sobre mi

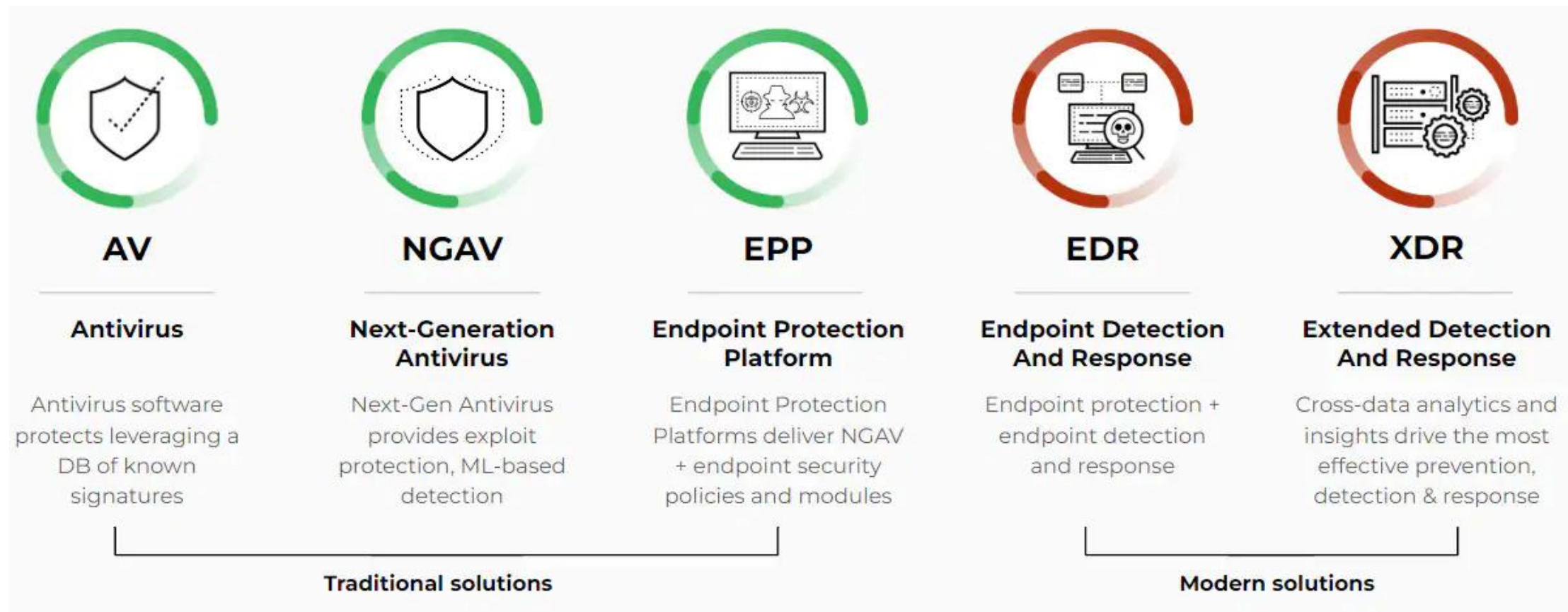
- **MANEL RODERO**

- Administrador de Sistemes, [Universitat Politècnica de Catalunya](#):
 - [UTGCNTIC](#) [2019 – actualitat]
 - [inLab FIB](#) [2012 – 2019]
 - LCFIB [1996 – 2012]
- Analista de Seguretat, [esCERT-UPC](#) [2013 – 2017]
- Enginyer Informàtic, [Facultat d'Informàtica de Barcelona](#) [1999]
- Contacte:
 - manel.rodero@upc.edu

Lab 4. AV → NGAV → EPP → EDR → XDR

Avaluar i comparar solucions de defensa segons criteris tècnics

Solucions tradicionals i modernes



Font: <https://www.paloaltonetworks.com/cyberpedia/what-is-an-endpoint-protection-platform-epp>

L'evolució de les defenses a l'endpoint

Eina	Enfocament principal	Limitació
Antivirus (AV)	Detecció per signatures (hash o patrons coneguts)	No detecta malware nou (zero-day)
NGAV (Next-Gen AV)	Afegeix heurística, ML i anàlisi de comportament	Encara centrat només en prevenció
EPP (Endpoint Protection Platform)	Suite completa: AV/NGAV + firewall + control de dispositius + xifrat	També centrat en la prevenció; poca visibilitat un cop l'atac ja hi és dins
EDR (Endpoint Detection & Response)	Monitorització contínua + capacitat de resposta (aïllar, investigar)	Només veu l'endpoint, no la resta de la infraestructura
XDR (eXtended Detection & Response)	Unifica telemetria d'endpoint + xarxa + núvol + identitat	Requereix més infraestructura i integració

Cronologia

The evolution of antivirus

1990s

Antivirus as the front line

Viruses spread through files and disks.

Signature-based detection caught most threats.

Antivirus tools were the primary layer of protection.

2010s

Limited detection role

Typical AV caught only a fraction of active malware.

Detection often came after infection

Cleanup became the main function.

Early 2000s

Attackers adapt

Malware authors tested code against AV before release.

Many new exploits bypassed signatures.

Antivirus effectiveness dropped against first-seen threats.

2020s

Evolution into EPP & EDR

AV shifted into broader endpoint protection platforms.

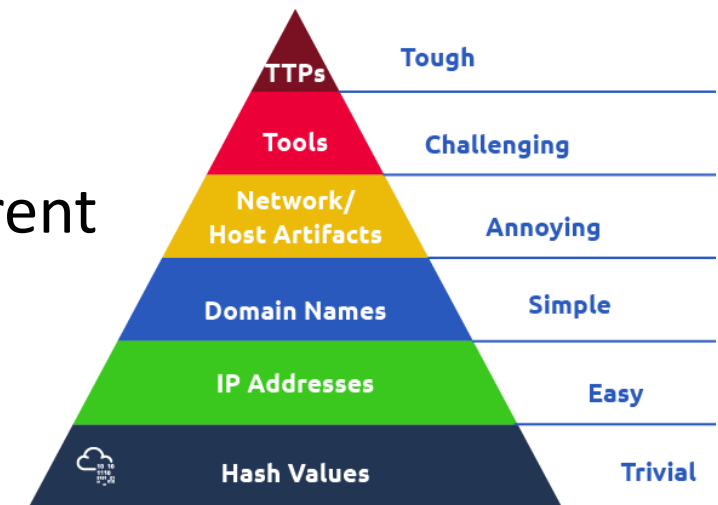
Focus on behavioral analysis, telemetry, and real-time monitoring.

Still useful for commodity threats, but part of a larger ecosystem.

Font: <https://www.paloaltonetworks.ca/cyberpedia/firewall-vs-antivirus>

Hash com a signatura

- El **hash** és l'**empremta digital** d'un fitxer
 - MD5/SHA1/SHA256
 - Si un fitxer té el mateix contingut → té el mateix hash
- AV clàssics: **comparació** amb llista de **hashes maliciosos**
- El hash és l'**indicador més dèbil** de la **Piràmide del Dolor**
 - Un sol byte canviat → hash completament diferent
 - S'evadeix fàcilment la detecció per signatura
- Necessitem **NGAV / EDR / XDR** per anar més enllà del hash



Font: <https://tryhackme.com/room/pyramidofpainax>

Comparant MD5 amb SHA256

COMPARING MD5 VS SHA-256 HASH FUNCTIONS

	MD5	SHA-256
# HASH LENGTH	 128-BIT / ALPHANUMERIC CHARS	 256-BIT / 64 ALPHANUMERIC CHARS
✓ SECURITY STATUS	 COLLISION-VULNERABLE	 CRYPTOGRAPHICALLY SECURE
⌚ COMPUTATION SPEED	 FAST	 MODERATE
📦 BEST USE CASE	LEGACY COMPATIBILITY	 FUTURE-PROOFING
# RECOMMENDATION	 BOTH TOGETHER FOR DEFENSE IN DEPTH	 BOTH TOGETHER FOR DEFENSE IN DEPTH

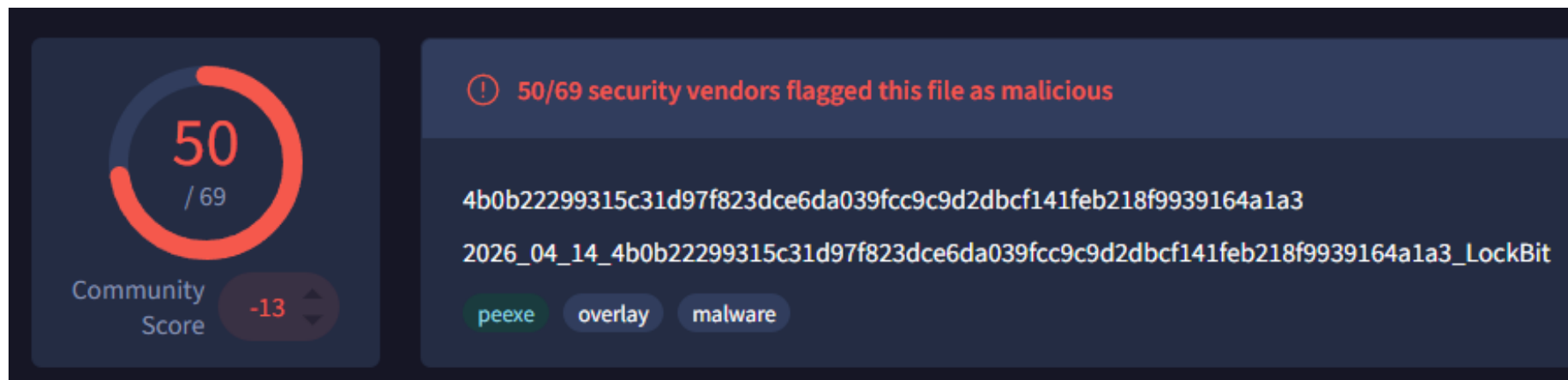
Font: <https://bdforensics.com/blog/md5-hashing-in-digital-forensics>

VirusTotal com a eina de triatge

- Plataforma que **agrega veredictes** de 60–70 motors AV
- Accepta **fitxers, URLs i hashes**
- Útil per **triatge ràpid**: veure si diversos motors detecten quelcom
- **No és un antivirus**: no protegeix en temps real
- Serveix per **contextualitzar** deteccions (quins motors, quins noms, quina prevalença)
- Ajuda a **prioritzar** anàlisi: sospitós o clarament maliciós
- Complementa l'analista, **no substitueix** NGAV/EDR/XDR

Interpretar els resultats de VirusTotal

Deteccions	Hash	Interpretació
0/70	No reconegut	Fitxer desconegut , no implica que sigui segur
0/70	Reconegut i amb historial	Probablement net (ja analitzat moltes vegades)
Poques deteccions (1-5/70)		Sospitós / possible fals positiu → cal contrastar
Moltes deteccions (>20/70)		Molt probablement maliciós



VirusTotal CLI (vt-cli)

- [VirusTotal CLI](#) és l'eina oficial de línia d'ordres que permet
 - Consultar l'API v3 de VirusTotal per analitzar fitxers o URL
 - [Es requereix una clau API](#)
 - Les claus gratuïtes tenen límits de velocitat i funcionalitats restringides
 - Obtenir informes
 - Cercar dades d'intel·ligència
 - Gestionar regles YARA a [LiveHunt](#)
 - Connectar-se al flux de fitxers analitzats per VT
 - Rebre notificacions quan algun d'ells coincideixi amb les nostres regles YARA
 - Executar tasques de [Retrohunt](#)
 - Escanejar els fitxers enviats durant els últims mesos amb les nostres regles YARA
 - 12 mesos per als usuaris de Hunting Pro; 3 mesos per als usuaris estàndard

Exercicis dirigits

- Demo A: Generar un fitxer i calcular-ne el hash

- Executar el següent codi:

```
echo "Missatge de prova" > fitxer_demo.txt  
sha256sum fitxer_demo.txt  
  
echo "missatge de prova" > fitxer_demo_2.txt  
sha256sum fitxer_demo_2.txt
```

- Demo B: Cercar el primer hash a VirusTotal

- No estarà. No vol dir que el fitxer sigui perillós sinó que ningú l'ha vist abans

- Demo C: Instal·lació i ús de ClamAV

- Analitzar fitxer_demo.txt i eicar.com
 - Diferències entre clamscan i clamdscan
 - Diferències entre ClamAV (motor local) i VirusTotal (agregador de motors)

Exercicis dirigits (ClamAV)

- ClamAV és un motor antivirus de codi obert
 - Detecta troians, virus, programari maliciós i altres amenaces malicioses

```
# Instal·lació
sudo apt Update && sudo apt install clamav -y
```

```
# Anàlisi (motor complet en mode autònom)
clamscan ./eicar.com ./fitxer_demo.txt
```

```
# Opcional
# Actualitzar la BD de signatures
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
```

```
# Instal·lació dimoni d'anàlisi
sudo apt install clamav-daemon -y
sudo systemctl start clamav-daemon
clamdsan eicar.com
```

```
alumne@ivysaur:~$ clamscan ./eicar.com ./fitxer_demo.txt
Loading: 7s, ETA: 0s [=====] 3.63M/3.63M sigs
Compiling: 2s, ETA: 0s [=====] 41/41 tasks

./eicar.com: Eicar-Test-Signature FOUND
./fitxer_demo.txt: OK

----- SCAN SUMMARY -----
Known viruses: 3628118
Engine version: 1.5.4
Scanned directories: 0
Scanned files: 2
Infected files: 1
Data scanned: 104 B
Data read: 86 B (ratio 1.21:1)
Time: 9.886 sec (0 m 9 s)
Start Date: 2026:10:04 19:29:05
End Date: 2026:10:04 19:29:15
alumne@ivysaur:~$ clamdsan ./eicar.com ./fitxer_demo.txt
/home/alumne/eicar.com: Eicar-Test-Signature FOUND
/home/alumne/./fitxer_demo.txt: OK

----- SCAN SUMMARY -----
Infected files: 1
Time: 0.024 sec (0 m 0 s)
Start Date: 2026:10:04 19:29:23
End Date: 2026:10:04 19:29:23
alumne@ivysaur:~$ |
```

Treball autònom en parelles



- El treball s'ha d'entregar en **1 únic fitxer PDF per grup** amb:
 - Fitxes completades
 - Comandes executades i resultat
- El nom del fitxer ha de tenir aquest format:
 - `Lab4_grupXX_cognom1_cognom2.pdf`
- L'entrega es fa mitjançant el **Racó**
- Teniu fins a la **mitjanit** del següent dimarts o divendres per entregar

Creació d'un "malware" benigne ;-)

- Compilar i executar un suposat "malware" escrit en C

- sudo apt install gcc
- gcc -o malware malware.c
- ./malware

- Comprovar si [ClamAV](#) el detecta

- clamscan ./malware

- Comprovar si [VT-CLI](#) el detecta

- Descarregar la versió 1.3.1+ per Linux x64
- ./vt init
- ./vt scan file ./malware --open

```
#include <stdio.h>
#include <string.h>

int main() {
    const char *cadena = "EICAR-LIKE-TEST-FIB-DA";

    printf("Simulant malware benigne per DA S4!\n");
    printf("Cadena (%zu): %s\n", strlen(cadena), cadena);
    return 0;
}
```

```
alumine@ivysaur:~$ ./malware
Simulant malware benigne per DA S4!
Cadena (22): EICAR-LIKE-TEST-FIB-DA
alumine@ivysaur:~$ clamscan ./malware
Loading:   5s, ETA:   0s [=====>] 3.63M/3.63M sigs
Compiling: 2s, ETA:   0s [=====>] 41/41 tasks

./malware: OK

----- SCAN SUMMARY -----
Known viruses: 3628118
Engine version: 1.5.4
Scanned directories: 0
Scanned files: 1
Infected files: 0
Data scanned: 15.67 KiB
Data read: 15.67 KiB (ratio 1.00:1)
Time: 8.188 sec (0 m 8 s)
Start Date: 2026:10:04 19:35:16
End Date: 2026:10:04 19:35:24
alumine@ivysaur:~$ |
```

Crear una signatura NDB (1 punt)

- Crear una signatura de ClamAV
 - Identificar el "malware" benigne
 - Fer-ho a binaris de tipus ELF
 - Anomenar-lo **EicarLikeTestFIBDA ;-)**
- Provar la signatura
 - de forma interactiva
 - copiant-la al directori /var/lib/clamav
- Cal entregar:
 - el contingut del fitxer de signatura
 - la sortida de clamscan

```

alumne@ivysaur:~/lab4$ clamscan -d malware.ndb ./malware
Loading: 0s, ETA: 0s [=====] 1/1 sigs
Compiling: 0s, ETA: 0s [=====] 40/40 tasks

./malware: EicarLikeTestFIBDA.UNOFFICIAL FOUND

----- SCAN SUMMARY -----
Known viruses: 1
Engine version: 1.5.4
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 15.67 KiB
Data read: 15.67 KiB (ratio 1.00:1)
Time: 0.017 sec (0 m 0 s)
Start Date: 2026:10:04 20:04:20
End Date: 2026:10:04 20:04:20
alumne@ivysaur:~/lab4$ sudo cp malware.ndb /var/lib/clamav/
alumne@ivysaur:~/lab4$ sudo systemctl restart clamav-daemon.service
alumne@ivysaur:~/lab4$ clamscan ./malware
Loading: 6s, ETA: 0s [=====] 3.63M/3.63M sigs
Compiling: 2s, ETA: 0s [=====] 41/41 tasks

./malware: EicarLikeTestFIBDA.UNOFFICIAL FOUND

----- SCAN SUMMARY -----
Known viruses: 3628119
Engine version: 1.5.4
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 15.67 KiB
Data read: 15.67 KiB (ratio 1.00:1)
Time: 8.139 sec (0 m 8 s)
Start Date: 2026:10:04 20:04:50
End Date: 2026:10:04 20:04:59
alumne@ivysaur:~/lab4$ |

```

Evacions encadenades (3 punts)

- Per cada evasió, modificar el codi del "malware" benigne perquè:
 - la signatura creada anteriorment ja NO el detecti
 - la sortida per pantalla sigui exactament la mateixa que el codi original
- **Evasió 1: Fragmentació en temps d'execució**
 - Dividir la cadena en variables separades i ajuntar-les en temps d'execució
- **Evasió 2: XOR amb clau fixa (0x5A)**
 - Cadena xifrada byte a byte amb la clau XOR i desxifrar-la a l'inici del main
- **Evasió 3: Inventeu la vostra tècnica d'evasió**
 - Expliqueu en una frase per què funciona
- **Cal entregar:**
 - Codi font modificat, sortida de clamscan i sortida del programa

Verificar amb gcc -O0 i gcc -O2
per evitar optimitzacions en
temps de compilació